

ESEMPIO

Prendiamo questa grammatica G_1 fatta in questo modo:

$$R \rightarrow S | T$$

$$S \rightarrow aSb | ab$$

$$T \rightarrow aTbb | abb$$

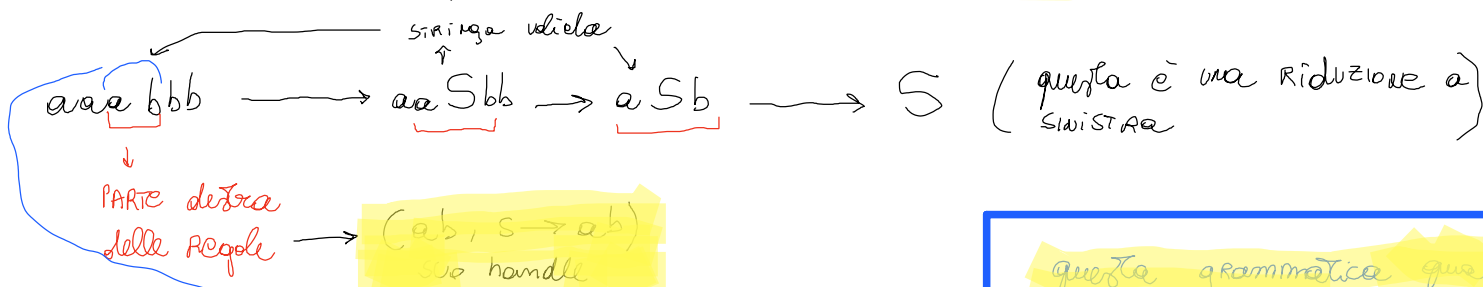
questa grammatica non è ambigua!

Non esistono due parse tree differenti, per arrivare ad una certa stringa ho un solo modo per procedere.

$$L(G_1) = B \cup C \quad \text{dove} \quad B = \{ a^n b^n \mid n \geq 0 \} \text{ e } C = \{ a^n b^{2n} \mid n \geq 0 \}$$

che alla fine dipende dove voglio espandere, o in S o in T all'inizio!

Prendiamo una stringa ed effettuiamo il processo di riduzione:



Prendiamo un'altra stringa valida:



questa grammatica non ha un handle forzato

- se come qui segue bb posso prendere ab come handle.
- se la stringa fosse $aaabbbbbbb$, l'handle da applicare sarebbe un altro, quindi, questa grammatica non è DCFL.

io all'inizio posso scegliere un'arbitrarietà di handle da applicare.

Ma la grammatica non è ambigua per questo, perché quelle sono riduzioni differenti, non parto dalla stessa stringa di terminali e arrivo alla stessa variabile iniziale!

Questa grammatica dà luogo ad un linguaggio CFL deterministico o no??

La macchina che deve riconoscere questo linguaggio (PDA) può fare a meno del determinismo oppure no?

Ho bisogno del non determinismo: Perché io conto le a , ma poi devo indovinare se confrontarle con le b o con le c .

Una volta che indovino non posso tornare indietro se mi rendo conto che non sono uguali, oramai lo stack l'ho consumato.

Qui il non determinismo è essenziale!

questa non è una DCFL, è una CFL! ✓

Se prendo la stessa grammatica ma con queste regole modificate:

$$R \rightarrow 1S | 2T$$

$$S \rightarrow aSb | ab$$

$$T \rightarrow aTbb | abb$$

$$L(G) = B \cup C$$

$$B = \{ a^m b^m \mid m \geq 0 \}$$

$$C = \{ a^m b^{2m} \mid m \geq 0 \}$$

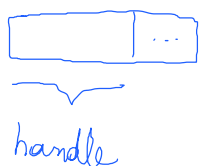
Bene, questa può essere deterministica: DCFL.

Perché?

Non c'è un non determinismo, non devo indovinare

Ogni riduzione di questa grammatica ha sempre "handle forzati"

Nella stringa valida, quando seleziono l'handle, è uguale, qualunque sia la stringa che segue, ovviamente se sono stringhe valide.



Infatti questa è una DCFL!